

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a549471bfa8c227b0.jsonl`  
- SHA-256: `cce0487dbb7c2a7a6e403bfb7fe5a682a17c07177470fc5fb706044d93090d31`  
- Source modified: `2026-07-08T06:18:59+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:16:23.555Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #2.

CLAIM (low/correctness) backend/pipeline/stitcher/compiler.py:31

summary: The NVENC availability probe does not use the same encoder args as the real per-clip encode, so it can report an encoder "runnable" that then fails on every actual clip with no fallback.

failure scenario: `_probe_encoder_available` (compiler.py:31) test-encodes with only `'-c:v h264_nvenc'` and no preset/quality flags, whereas `_select_encoder` (compiler.py:294-297) runs every real clip with `'-c:v h264_nvenc -preset p4 -cq 23 -pix_fmt yuv420p'`. On an ffmpeg build where basic NVENC initialises but the p1-p7 preset naming (`'-preset p4'`) is unsupported (older nvenc preset scheme), the probe returns True, the encoder is committed once for the whole run, then every `_trim_one_segment` encode fails; `temp_clips` ends up empty and `compile_highlights` raises `RuntimeError` -> the reel fails entirely, when a probe that matched the real command would have fallen back to `libx264` and produced a reel. The probe's stated contract (`'a real test encode ... distinguishes usable from will-fail-at-runtime'`) is not met. Version-gated (modern ffmpeg on the L4 image is unaffected), hence low confidence.

2. assistant (2026-07-08T06:16:31.238Z)

I'll investigate this claim by reading the actual code. The context paths show "undefined" so let me find the real files in the repository.

3. user (2026-07-08T06:16:37.280Z)

dac6736 `perf(serving)`: run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)
bf97700 `docs(runlogs)`: record live SPA CUJ closing the 2 owner-gated validation-latency cells (§9) (#522)
699d85b `feat(validation)`: cache validation verdict by `video_id` to skip re-validation on reupload (#518)
71c88ef `docs(runlogs)`: validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d `docs(runlogs)`: record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 `fix(cloud-run)`: fail fast when a worker execution terminates without a done-marker (#515)
f5c9dd9 `chore(ui)`: re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 `fix(serving)`: fail fast when `compute=local` forces weightless in-process detection (#511)
20862d7 `perf(validators)`: decode `scene_cut/blur/relevance` frames in one shared pass (#512)
23c1868 `fix(serving)`: worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 `docs(runlog)`: DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad `chore(serving)`: forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 `feat(serving)`: default cloud-serving to `nvdec_resident` decode (#508)
44a2f47 `feat(detectors)`: GPU-resident WASB NVDEC decode behind default-OFF `decode_backend` flag (#507)
17d844b `docs(runlog)`: alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed + downsample fix proven (#504)

4. user (2026-07-08T06:16:38.209Z)

`./backend/pipeline/stitcher/compiler.py`

5. assistant (2026-07-08T06:16:44.254Z)

The code is merged. Let me read the `compiler.py` and the actual diff from the merge commit.

6. user (2026-07-08T06:16:45.489Z)

```
1     import logging
2     import os
3     import subprocess
4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10         ffmpeg_timeout_sec,
```

```

11         ffmpeg_bin,
12         ffmpeg_timeout_sec,
13         timeout_label,
14     )
15
16     logger = logging.getLogger(__name__)
17
18     # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19     # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20     # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21     # backend/pipeline/stitcher/, so parents[2] == backend/.
22     _BUNDLED_FONT = (
23         Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24     )
25
26     #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
27     #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
28     _ENCODER_AVAILABLE: Dict[str, bool] = {}
29
30
31     def _probe_encoder_available(encoder: str) -> bool:
32         """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
33         lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
34         real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
35         runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
36         if encoder in _ENCODER_AVAILABLE:
37             return _ENCODER_AVAILABLE[encoder]
38         ok = False
39         # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
ffmpeg
40         # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
41         configured = ffmpeg_timeout_sec()
42         probe_timeout = min(30.0, float(configured)) if configured else 30.0
43         try:
44             proc = subprocess.run(
45                 [
46                     ffmpeg_bin(),
47                     "-hide_banner",
48                     "-f",
49                     "lavfi",
50                     "-i",
51                     "color=c=black:s=64x64:d=0.1",
52                     "-frames:v",
53                     "1",
54                     "-c:v",
55                     encoder,
56                     "-f",
57                     "null",
58                     "-",
59                 ],
60                 capture_output=True,
61                 timeout=probe_timeout,
62             )
63             ok = proc.returncode == 0
64         except Exception: # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
fall back"

```

```

65         ok = False
66         _ENCODER_AVAILABLE[encoder] = ok
67         return ok
68
69
70     class VideoCompiler:
71         def __init__(
72             self,
73             output_dir: str,
74             end_padding: float = 1.0,
75             begin_padding: float = 0.5,
76             watermark: Optional[Any] = None,
77             encoder: str = "libx264",
78             encode_profile: Optional[Any] = None,
79         ):
80             self.output_dir = output_dir
81             self.end_padding = end_padding
82             self.begin_padding = begin_padding
83             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
84             # normalized to a dict (or None) so this module stays config-package-agnostic.
85             self.watermark = self._normalize_watermark(watermark)
86             # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
87             # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
88             # re-encode the watermark forces. The requested encoder is PROBED once per run
89             # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
90             self.encoder = (encoder or "libx264").strip() or "libx264"
91             self.encode_profile = self._normalize_encode_profile(encode_profile)
92             os.makedirs(output_dir, exist_ok=True)
93
94         @staticmethod
95         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
96             if watermark is None:
97                 return None
98             if hasattr(watermark, "model_dump"):
99                 return watermark.model_dump()
100             if isinstance(watermark, dict):
101                 return dict(watermark)
102             return None
103
104         @staticmethod
105         def _normalize_encode_profile(profile: Optional[Any]) -> Dict[str, Any]:
106             if profile is None:
107                 raw: Dict[str, Any] = {}
108             elif hasattr(profile, "model_dump"):
109                 raw = profile.model_dump()
110             elif isinstance(profile, dict):
111                 raw = dict(profile)
112             else:
113                 raw = {}
114             return {
115                 "max_width": int(raw.get("max_width") or 0),
116                 "max_height": int(raw.get("max_height") or 0),
117                 "crf": int(raw.get("crf", 22)),
118                 "audio_bitrate": str(raw.get("audio_bitrate") or "128k"),
119                 "video_preset": str(raw.get("video_preset") or "superfast"),
120                 "pix_fmt": str(raw.get("pix_fmt") or ""),
121             }
122
123         @staticmethod
124         def _ff_quote(path: str) -> str:
125             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
126             filtergraph operand (drawtext fontfile/textfile, movie= source): forward
127             slashes (Windows-friendly), escaped single quotes, AND escaped colons.

```

```

128
129     The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
130     option parser treats ':' as the option separator, so a Windows drive-letter
131     path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
132     the encode fails with "Invalid argument". This silently disabled the
133     on-by-default bundled-font watermark (and absolute logo paths) on Windows.""
134     return path.replace("\\", "/").replace("'", r"\').replace(":", r"\:")
135
136     @staticmethod
137     def _parse_time(val: Union[int, float, str]) -> float:
138         """Normalizes a timestamp value to float seconds.
139         Handles: float/int (pass-through), plain numeric strings ("12.4"),
140         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
141         This mirrors the parse_time logic used in gemini.py to ensure
142         the stitcher can always consume whatever format the indexer produced."""
143         if isinstance(val, (int, float)):
144             return float(val)
145         if isinstance(val, str):
146             val = val.strip()
147             if ":" in val:
148                 # Handle MM:SS, HH:MM:SS, etc.
149                 parts = val.split(":")
150                 parts.reverse()
151                 total = 0.0
152                 for i, p in enumerate(parts):
153                     try:
154                         total += float(p) * (60**i)
155                     except ValueError:
156                         pass
157                 return total
158             try:
159                 return float(val)
160             except ValueError:
161                 logger.warning(
162                     f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
163                 )
164                 return 0.0
165             logger.warning(
166                 f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
167             )
168             return 0.0
169
170     @staticmethod
171     def _merge_overlapping(
172         segments: List[Tuple[float, float]],
173     ) -> List[Tuple[float, float]]:
174         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
175
176         A highlight reel must never replay the same footage. The same rally can land in
177         the segment list more than once ? e.g. two detector runs accumulated for one
178         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
179         or
180         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
181         rally
182         twice. Merging any range that overlaps or abuts the previous one collapses true
183         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
184         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
185         """
186         parsed = []
187         for raw_s, raw_e in segments:
188             s = VideoCompiler._parse_time(raw_s)
189             e = VideoCompiler._parse_time(raw_e)
190             if e > s:

```

```

189         parsed.append((s, e))
190     parsed.sort()
191     merged: List[Tuple[float, float]] = []
192     for s, e in parsed:
193         if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
194             merged[-1] = (merged[-1][0], max(merged[-1][1], e))
195         else:
196             merged.append((s, e))
197     return merged
198
199 def _get_video_duration(self, video_path: str) -> float:
200     """Probes the video file to get its total duration in seconds.
201     Returns 0.0 if it cannot be determined."""
202     try:
203         timeout = ffprobe_timeout_sec()
204         result = subprocess.run(
205             [
206                 ffprobe_bin(),
207                 "-v",
208                 "error",
209                 "-show_entries",
210                 "format=duration",
211                 "-of",
212                 "default=noprint_wrappers=1:nokey=1",
213                 video_path,
214             ],
215             capture_output=True,
216             text=True,
217             check=True,
218             timeout=timeout,
219         )
220         return float(result.stdout.strip())
221     except subprocess.TimeoutExpired:
222         logger.warning(
223             "Could not determine video duration via ffprobe: timed out after %s",
224             timeout_label(ffprobe_timeout_sec()),
225         )
226         return 0.0
227     except Exception as e:
228         logger.warning(f"Could not determine video duration via ffprobe: {e}")
229         return 0.0
230
231 def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
232     """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
233     and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
234     watermarking is disabled or has nothing to draw. The concat stage stays -c copy
235     because the mark is already baked into every clip identically."""
236     wm = self.watermark
237     if not wm or not wm.get("enabled"):
238         return None
239
240     text = (wm.get("text") or "").strip()
241     logo_path = (wm.get("logo_path") or "").strip()
242     if logo_path and not os.path.exists(logo_path):
243         logger.warning(
244             f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
245         )
246         logo_path = ""
247     if not text and not logo_path:
248         return None
249
250     margin = int(wm.get("margin", 24))
251     opacity = float(wm.get("opacity", 0.85))
252     position = str(wm.get("position", "bottom-right"))
253     # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +

```

```

w/h (logo).
254         dt_xy = {
255             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
256             "bottom-left": f"x={margin}:y=h-th-{margin}",
257             "top-right": f"x=w-tw-{margin}:y={margin}",
258             "top-left": f"x={margin}:y={margin}",
259         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
260         ov_xy = {
261             "bottom-right": f"W-w-{margin}:H-h-{margin}",
262             "bottom-left": f"{margin}:H-h-{margin}",
263             "top-right": f"W-w-{margin}:{margin}",
264             "top-left": f"{margin}:{margin}",
265         }.get(position, f"W-w-{margin}:H-h-{margin}")
266
267         drawtext = ""
268         if text:
269             ratio = float(wm.get("font_size_ratio") or 0.035)
270             divisor = max(1, round(1.0 / ratio))
271             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
272             tf_path = os.path.join(tmp_dir, "watermark.txt")
273             with open(tf_path, "w", encoding="utf-8") as fh:
274                 fh.write(text)
275             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
276             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
277             font_path = (wm.get("font_path") or "").strip()
278             if not font_path and _BUNDLED_FONT.exists():
279                 font_path = str(_BUNDLED_FONT)
280             if font_path:
281                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
282             else:
283                 font_opt = f"font='{wm.get('font', 'Sans')}'"
284             box = ""
285             if wm.get("box", True):
286                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
287             drawtext = (
288                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
289                 f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
290             )
291
292         if logo_path and drawtext:
293             return
294         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
295         if logo_path:
296             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
297         return drawtext
298
299     def _scale_filter(self) -> Optional[str]:
300         max_width = int(self.encode_profile.get("max_width") or 0)
301         max_height = int(self.encode_profile.get("max_height") or 0)
302         filters = []
303         if max_width > 0 and max_height > 0:
304             filters.append(
305                 f"scale=w='{min({max_width}, iw)':h='{min({max_height}, ih)}'"
306                 f":force_original_aspect_ratio=decrease:force_divisible_by=2"
307             )
308         pix_fmt = str(self.encode_profile.get("pix_fmt") or "").strip()
309         if pix_fmt:
310             filters.append(f"format={pix_fmt}")
311         return ",".join(filters) if filters else None
312
313     @staticmethod
314     def _combine_video_filters(
315         scale_filter: Optional[str], watermark_filter: Optional[str]

```

```

315         ) -> Optional[str]:
316             if not scale_filter:
317                 return watermark_filter
318             if not watermark_filter:
319                 return scale_filter
320             if watermark_filter.startswith("movie=") and "[in][wm]" in watermark_filter:
321                 return (
322                     f"[in]{scale_filter}[scaled];"
323                     + watermark_filter.replace("[in][wm]", "[scaled][wm]", 1)
324                 )
325             return f"{scale_filter},{watermark_filter}"
326
327     def _select_encoder(self) -> Tuple[List[str], str]:
328         """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
329         chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and
the final
330         concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode;
if the
331         requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to "
332         libx264 so a misconfig never nukes the reel."""
333         enc = self.encoder
334         if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
335             logger.warning(
336                 "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
337                 "libx264 (CPU). On the L4 worker this should be available.",
338                 enc,
339             )
340             enc = "libx264"
341         if enc == "libx264":
342             return [
343                 "-c:v",
344                 "libx264",
345                 "-preset",
346                 str(self.encode_profile["video_preset"]),
347                 "-crf",
348                 str(self.encode_profile["crf"]),
349             ], enc
350         # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
351         # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
352         cq = "23"
353         if int(self.encode_profile.get("max_width") or 0) > 0 and int(
354             self.encode_profile.get("max_height") or 0
355         ) > 0:
356             cq = str(self.encode_profile["crf"])
357         pix_fmt = str(self.encode_profile.get("pix_fmt") or "yuv420p")
358         return (
359             ["-c:v", enc, "-preset", "p4", "-cq", cq, "-pix_fmt", pix_fmt],
360             enc,
361         )
362
363     def _trim_one_segment(
364         self,
365         idx: int,
366         raw_start: Union[int, float, str],
367         raw_end: Union[int, float, str],
368         segments: List[Tuple[float, float]],
369         video_duration: float,
370         tmp_dir: str,
371         raw_video_path: str,
372         video_filter: Optional[str],
373         watermark_fallback_filter: Optional[str],

```

```

374         video_codec_args: Optional[List[str]] = None,
375     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
376         """Trim a single segment into its own clip file (extracted verbatim from the
377         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
378         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
379         None and the (idx, start, end, reason) record to record in skipped_segments."""
380         # Normalize timestamps to float seconds ? handles float, int,
381         # plain numeric strings, and colon-separated formats like "MM:SS"
382         start = self._parse_time(raw_start)
383         end = self._parse_time(raw_end)
384         segment_label = (
385             f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
386         )
387
388         # Validate the segment times
389         if start < 0:
390             logger.warning(
391                 f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
392                 0."
393             )
394             start = 0.0
395
396         if start >= end:
397             logger.error(
398                 f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
399                 Skipping invalid segment."
400             )
401             return None, (idx, start, end, "start >= end")
402
403         # Check if segment extends beyond video duration
404         if video_duration > 0:
405             if start >= video_duration:
406                 logger.error(
407                     f"{segment_label}: start_time ({start:.2f}s) is beyond the video
408                     duration ({video_duration:.2f}s). "
409                     f"This segment cannot be extracted ? the video ran out. Skipping."
410                 )
411                 return None, (idx, start, end, "start beyond video end")
412
413             if end > video_duration:
414                 original_end = end
415                 end = video_duration
416                 logger.warning(
417                     f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
418                     duration ({video_duration:.2f}s). "
419                     f"Clamping end_time to {video_duration:.2f}s. The segment may be
420                     truncated."
421                 )
422
423         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
424
425         # Precise sub-second trim using fast re-encoding to preserve stream headers
426         padded_start = max(0.0, start - self.begin_padding)
427         padded_end = end + self.end_padding
428
429         # Clamp padded_end to video duration if known
430         if video_duration > 0 and padded_end > video_duration:
431             padded_end = video_duration
432             logger.info(
433                 f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
434                 video duration. "
435                 f"Clamping to {video_duration:.2f}s (end padding partially applied)."
436             )
437
438         trim_duration = padded_end - padded_start

```

```

433         logger.info(
434             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
435         )
436
437     # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
438     # directly without resolving an encoder ? keeps this method's old behaviour
intact.
439     codec_args = (
440         video_codec_args
441         if video_codec_args is not None
442         else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
443     )
444     base_cmd = [
445         ffmpeg_bin(),
446         "-y",
447         "-ss",
448         str(round(padded_start, 3)),
449         "-to",
450         str(round(padded_end, 3)),
451         "-i",
452         raw_video_path,
453         *codec_args,
454         "-c:a",
455         "aac",
456         "-b:a",
457         str(self.encode_profile["audio_bitrate"]),
458     ]
459
460     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
461     # path), retry once WITHOUT it so a branding misconfig can never nuke the
462     # whole reel. If a scale filter is active (SD rendition), the fallback keeps
463     # that scale filter so SD cannot silently become an original-size reel.
464     # Clips stay codec-identical either way, so concat -c copy holds.
465     if video_filter:
466         attempts: List[Tuple[Optional[str], bool]] = [(video_filter, False)]
467         if watermark_fallback_filter != video_filter:
468             attempts.append((watermark_fallback_filter, True))
469     else:
470         attempts = [(None, False)]
471     produced = False
472     last_exc: Optional[subprocess.CallProcessError] = None
473     cmd = base_cmd + [clip_path]
474     timeout = ffmpeg_timeout_sec()
475     for attempt_filter, without_watermark in attempts:
476         attempt_vf = ["-vf", attempt_filter] if attempt_filter else []
477         cmd = base_cmd + attempt_vf + [clip_path]
478         try:
479             subprocess.run(
480                 cmd, capture_output=True, check=True, timeout=timeout
481             )
482         except subprocess.TimeoutExpired:
483             reason = f"ffmpeg timeout after {timeout_label(timeout)}"
484             logger.error(f"{segment_label}: Ffmpeg trim timed out.")
485             print(f"  Command: {' '.join(cmd)}")
486             print(f"  {reason}")
487             return None, (idx, start, end, reason)
488         except subprocess.CallProcessError as e:
489             last_exc = e
490             if not without_watermark and len(attempts) > 1:
491                 _err = (
492                     e.stderr.decode(errors="replace").strip()[-300:]
493                     if getattr(e, "stderr", None)
494                     else ""

```

```

495         )
496         logger.warning(
497             f"{segment_label}: watermark encode failed; retrying without it.
498 "
499             f"ffmpeg: {_err or '(no stderr captured)'}"
500         )
501         continue
502     break
503     if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
504         produced = True
505         if without_watermark and len(attempts) > 1:
506             logger.warning(
507                 f"{segment_label}: encoded WITHOUT the watermark (the watermark
508 filter failed)."
509             )
510             break
511         last_exc = None # ffmpeg succeeded but produced nothing
512         break
513     if produced:
514         return clip_path, None
515     elif last_exc is not None:
516         stderr_msg = last_exc.stderr.decode(errors="replace").strip()
517         logger.error(f"{segment_label}: FFmpeg trim failed.")
518         print(f"  Command: {' '.join(cmd)}")
519         print(
520             f"    FFmpeg stderr: {stderr_msg[-500:]}")
521         ) # Last 500 chars to avoid flooding
522         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
523     else:
524         logger.error(
525             f"{segment_label}: FFmpeg exited successfully but output clip is missing
526 or empty. Skipping."
527         )
528         return None, (idx, start, end, "empty output clip")
529
530 def compile_highlights(
531     self,
532     raw_video_path: str,
533     segments: List[Tuple[float, float]],
534     output_filename: str,
535 ) -> str:
536     """
537     Extracts selected segments from a raw video and stitches them together
538     into a seamless highlights file, applying end_padding to prevent abrupt endings.
539
540     .. warning::
541         This method uses a two-stage codec contract: per-clip re-encodes must use
542         IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
543         concat can stream-copy (``-c copy``) without re-encoding. Changing the
544         clip encode settings without updating the concat assumptions will produce
545         broken output.
546     """
547     if not segments:
548         raise ValueError("No highlight segments provided to compile.")
549
550     # Never stitch the same footage twice: collapse overlapping/duplicate ranges
551     # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
552     original_count = len(segments)
553     segments = self._merge_overlapping(segments)
554     if len(segments) < original_count:
555         logger.info(
556             "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
557 %d.",
558             original_count - len(segments),

```

```

556         original_count,
557         len(segments),
558     )
559     if not segments:
560         raise ValueError("No valid (start < end) highlight segments after merge.")
561
562     if not os.path.exists(raw_video_path):
563         raise FileNotFoundError(
564             f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
565         )
566
567     # Defense-in-depth: never let the output name escape output_dir (the API also
568     # validates this at the request boundary).
569     output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
570
571     # Probe video duration so we can detect out-of-range segments
572     video_duration = self._get_video_duration(raw_video_path)
573     if video_duration > 0:
574         logger.info(f"Source video duration: {video_duration:.2f}s")
575     else:
576         logger.warning(
577             "Video duration unknown. Cannot validate segment boundaries against
578             video length."
579         )
580
581     # We will create temporary clips and stitch them using FFmpeg concat demuxer
582     temp_clips = []
583     skipped_segments = []
584
585     # Create a temp directory within output_dir
586     with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
587         logger.info(
588             f"Trimming {len(segments)} segments
589             (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
590         )
591         # Build the (optional) watermark filtergraph once; it is applied during
592         every
593         # per-clip re-encode so the mark is baked into each clip identically.
594         scale_filter = self._scale_filter()
595         watermark_filter = self._watermark_filter(tmp_dir)
596         video_filter = self._combine_video_filters(scale_filter, watermark_filter)
597         watermark_fallback_filter = scale_filter if watermark_filter else
598         video_filter
599         if scale_filter:
600             logger.info("[rendition] applying video filter: %s", scale_filter)
601         if watermark_filter:
602             logger.info("[watermark] applying branding overlay to each clip.")
603         # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
604         shares one codec
605         # and the final concat stays -c copy (#521).
606         video_codec_args, chosen_encoder = self._select_encoder()
607         if chosen_encoder != "libx264":
608             logger.info("[compile] encoding clips with %s (hardware).",
609                 chosen_encoder)
610
611         for idx, (raw_start, raw_end) in enumerate(segments):
612             clip_path, skip_record = self._trim_one_segment(
613                 idx,
614                 raw_start,
615                 raw_end,
616                 segments,
617                 video_duration,
618                 tmp_dir,
619                 raw_video_path,
620                 video_filter,
621                 watermark_fallback_filter,

```

```

615         video_codec_args,
616     )
617     if clip_path is not None:
618         temp_clips.append(clip_path)
619     if skip_record is not None:
620         skipped_segments.append(skip_record)
621
622     # Report summary of skipped segments
623     if skipped_segments:
624         logger.info(
625             f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
626         )
627         for seg_idx, seg_start, seg_end, reason in skipped_segments:
628             logger.info(
629                 f"    - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
630             )
631             logger.info("")
632
633     if not temp_clips:
634         raise RuntimeError(
635             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
636             f"All {len(segments)} segments were skipped. "
637             f"Check the errors above for details. "
638             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
639         )
640
641     logger.info(
642         f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
643     )
644
645     # Create a list file for FFmpeg concat demuxer
646     list_file_path = os.path.join(tmp_dir, "clips.txt")
647     # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
648
649     # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
650     with open(list_file_path, "w", encoding="utf-8") as f:
651         for clip in temp_clips:
652             # The concat demuxer resolves relative entries against the list
file's
653             # directory. Use absolute paths so a temp-dir clip does not become
654             # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
655             safe_path = Path(clip).resolve().as_posix().replace("'", "'\\'")
656             f.write(f"file '{safe_path}'\n")
657
658     logger.info("Stitching clips together...")
659     # Stitch them together without re-encoding (since they were encoded with
identical settings)
660     concat_cmd = [
661         ffmpeg_bin(),
662         "-y",
663         "-f",
664         "concat",
665         "-safe",
666         "0",
667         "-i",
668         list_file_path,
669         "-c",
670         "copy",
671         output_path,
672     ]

```

```

673         try:
674             timeout = ffmpeg_timeout_sec()
675             subprocess.run(
676                 concat_cmd, capture_output=True, check=True, timeout=timeout
677             )
678
679             # Final verification
680             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
681                 raise RuntimeError(
682                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
683                     f"but output file is missing or empty: {output_path}"
684                 )
685
686             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
687             logger.info(
688                 f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
689 MB)"
690             )
691             return output_path
692         except subprocess.CalledProcessError as e:
693             stderr_msg = e.stderr.decode(errors="replace").strip()
694             raise RuntimeError(
695                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
696                 f"  Command: {' '.join(concat_cmd)}\n"
697                 f"  FFmpeg stderr: {stderr_msg[-500:]}"
698             ) from e
699         except subprocess.TimeoutExpired as e:
700             raise RuntimeError(
701                 f"[COMPILER ERROR] FFmpeg concatenation timed out after
702 {timeout_label(timeout)}.\n"
703                 f"  Command: {' '.join(concat_cmd)}"
704             ) from e

```

7. assistant (2026-07-08T06:17:40.613Z)

Now let me examine the deployment context ? the L4 worker image's ffmpeg version ? since the claim hinges on an old ffmpeg build where NVENC initializes but `-preset p4` is rejected.

8. user (2026-07-08T06:17:45.289Z)

```

commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:29:33 2026 +0530

```

perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement (#521) (#525)

The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the 2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode with no NVENC). Validation (183s) also runs there; both serialize on `_LOCAL_COMPUTE_LANE`, so one run pins the control-plane ~16 min ? won't scale past a handful of concurrent users.

- Config-driven placement: `deployment.phase_placement {validation|segment|compile} ? control_plane | cloud_run_l4`, resolved by `backend/config/placement.py`. Unset ? today's placement, byte-identical. A config-load guard rejects the "nobody validates" footgun (validation on L4 without a cloud segment).
- compile ? L4: a `kind='compile'` Cloud Run dispatch mirroring `_maybe_dispatch_remote`. The control-plane resolves the reel's segments, stages a compile-spec (resolved time-pairs + stitching config + source uri) to the bucket, dispatches ``worker compile`` on the L4, bucket-polls a done-marker, and returns the same `{status, filepath, download_url}` shape the SPA expects. The worker stitches via the

- SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
- NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
- validation delegation: phase_placement.validation moves the validator suite off the control-plane via the existing --skip-validation lever (#513), guarded so an in-process run always validates.
- cloud-serving preset places compile on the L4.

Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).

Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to test_compute_backend.py (run_compile), test_worker.py (cmd_compile), test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

backend/compute/__init__.py	2 +
backend/compute/base.py	34 +++
backend/compute/cloud_run.py	100 +++++-
backend/config/__init__.py	16 ++
backend/config/models.py	52 ++++
backend/config/placement.py	59 ++++
backend/config/presets.py	10 +-
backend/orchestration.py	322 ++++++-----
backend/pipeline/stitcher/compiler.py	94 +++++-
backend/worker/__main__.py	139 +++++++
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md	169 +++++++
docs/COMPUTE_DECOUPLED_SERVING/README.md	1 +
tests/test_api_cloud_dispatch.py	70 +++++
tests/test_compile_dispatch.py	204 +++++++
tests/test_compiler.py	64 +++++
tests/test_compute_backend.py	111 +++++++
tests/test_config_deployment.py	5 +
tests/test_phase_placement.py	138 +++++++
tests/test_worker.py	175 +++++++

9. user (2026-07-08T06:17:45.545Z)

```
deploy\gcp\nightly_regression\Dockerfile.launcher
deploy\cloudrun\Dockerfile
```

10. assistant (2026-07-08T06:17:51.357Z)

Let me check the L4 worker image's ffmpeg provenance and the new encoder-selection tests.

11. user (2026-07-08T06:17:52.886Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
```

```

11      # * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506
modernized stack,
12      # validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs
the WASB
13      # shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON),
so the app's
14      # torch never co-installs with it.
15      #
16      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
17      # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
18      # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
19
20      # Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver
stack but not
21      # the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and
PyNvVideoCodec needs
22      # BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
23      # driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU
fleet's driver
24      # (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
25      # so a large version skew can break decode ? the golden gate on the real image is the
check.
26      ARG NVIDIA_DRIVER_VERSION=580.159.03
27      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
28      ARG NVIDIA_DRIVER_VERSION
29      RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
30          && rm -rf /var/lib/apt/lists/* \
31          && curl -fsSL -o /tmp/drv.run \
32              "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
33          && sh /tmp/drv.run --extract-only --target /tmp/drv \
34          && mkdir -p /nvlibs \
35          && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
36          && rm -rf /tmp/drv /tmp/drv.run
37
38      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
39
40      ENV DEBIAN_FRONTEND=noninteractive \
41          PYTHONUNBUFFERED=1 \
42          PIP_NO_CACHE_DIR=1 \
43          CONDA_DIR=/opt/conda \
44          WASB_REPO_DIR=/opt/models/WASB-SBDT
45
46      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
47      RUN apt-get update && apt-get install -y --no-install-recommends \
48          ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
49          python3 python3-pip python3-venv \
50          && rm -rf /var/lib/apt/lists/*
51
52      # NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
53      # PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
54      ARG NVIDIA_DRIVER_VERSION
55      COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
56      RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
57          && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
58          && ldconfig
59

```

```

60      # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
61      # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
62      # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
63      # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
64      # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
65      # is UNCHANGED.
66      #
67      # #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
68      # (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
69      # PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
70      # not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version
shims this bump
71      # needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
72      # UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
73      # the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
74      ENV MAMBA_ROOT_PREFIX=/opt/conda
75      RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
76          | tar -xj -C /usr/local/bin/micromamba \
77          && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
78          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
79          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
80          torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
81          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec==2.1.0 \
82          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
83          hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
84
85      # --- Main application env ---
86      WORKDIR /app
87      COPY pyproject.toml README.md ./
88      COPY requirements-trackers.txt ./
89      COPY backend ./backend
90      COPY training ./training
91      # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
92      # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
93      # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
94      # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
95      # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
96      # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
97      # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
98      # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
99      # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
100     # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane

```

```

101     # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
102     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
103     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
104     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
105     # single-sourced in requirements-trackers.txt.
106     RUN python3 -m pip install --upgrade pip \
107         && python3 -m pip install -r requirements-trackers.txt \
108         && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
109
110     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
111     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
112     # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
113     # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
114     # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
115     ENV PYTHONUNBUFFERED=1 \
116         WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
117         WASB_REPO_DIR=/opt/models/WASB-SBDT \
118         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
119         RALLY_DETECTOR_IMPL=native
120
121     # The dispatcher (CloudRunCompute) supplies the per-execution args:
122     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
123     ENTRYPOINT ["python3", "-m", "backend.worker"]
124

```

12. user (2026-07-08T06:17:53.589Z)

```

1     """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2     where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3
4     import os
5     import subprocess
6     from unittest.mock import MagicMock, patch
7
8     import pytest
9
10    from backend.pipeline.stitcher.compiler import VideoCompiler
11
12
13    def test_parse_time_formats():
14        f = VideoCompiler._parse_time
15        assert f(12.5) == 12.5
16        assert f("12.5") == 12.5
17        assert f("1:30") == 90.0
18        assert f("1:00:00") == 3600.0
19        assert f("garbage") == 0.0
20
21
22    def test_merge_overlapping_collapses_duplicates_and_overlaps():
23        m = VideoCompiler._merge_overlapping
24        # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
25        assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
26            (28.2, 34.5),
27            (40.5, 45.5),
28        ]

```

```

29     # Unsorted input is sorted; an overlapping range extends the previous.
30     assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
31         (0.0, 5.0),
32         (100.0, 120.0),
33     ]
34     # Genuinely distinct (gapped) rallies are preserved, not merged.
35     assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
36     # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
37     assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]
38
39
40 def test_compile_dedups_before_stitching(tmp_path):
41     """A segment list with every rally twice must yield ONE clip per unique rally."""
42     comp = VideoCompiler(str(tmp_path / "out"))
43     raw = tmp_path / "raw.mp4"
44     raw.write_bytes(b"x")
45     trims = []
46
47     def fake_run(cmd, *a, **k):
48         if cmd[0] == "ffprobe":
49             return MagicMock(stdout="100.0\n", returncode=0)
50         out = cmd[-1]
51         if isinstance(out, str) and out.endswith(".mp4"):
52             open(out, "wb").write(b"clip")
53             # count per-clip trims (the concat list file input is not an .mp4 output)
54             if "-i" in cmd and cmd.index("-i") + 1 == str(raw):
55                 trims.append(cmd)
56             return MagicMock(returncode=0, stdout=b"", stderr=b"")
57
58     dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
59     with patch.object(subprocess, "run", side_effect=fake_run):
60         comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
61     # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
62     assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"
63
64
65 def test_compile_basenamees_output_and_stitches(tmp_path):
66     comp = VideoCompiler(str(tmp_path / "out"))
67     raw = tmp_path / "raw.mp4"
68     raw.write_bytes(b"x")
69
70     calls = []
71
72     def fake_run(cmd, *a, **k):
73         calls.append(cmd)
74         # Create whatever output file ffmpeg/ffprobe was told to write so existence
75         if cmd[0] == "ffprobe":
76             return MagicMock(stdout="100.0\n", returncode=0)
77         out = cmd[-1]
78         if isinstance(out, str) and out.endswith(("mp4",)):
79             open(out, "wb").write(b"clip")
80             return MagicMock(returncode=0, stdout=b"", stderr=b"")
81
82     with patch.object(subprocess, "run", side_effect=fake_run):
83         # Traversal in the name must be stripped (defense-in-depth basename).
84         out_path = comp.compile_highlights(
85             str(raw), [(0.0, 5.0), (10.0, 15.0)], "../reel.mp4"
86         )
87
88     assert os.path.basename(out_path) == "reel.mp4"
89     assert os.path.dirname(out_path).endswith("out")
90     # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
91     assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3
92

```

```

93
94     def test_compile_original_profile_keeps_historical_encode_args(tmp_path):
95         comp = VideoCompiler(str(tmp_path / "out"))
96         raw = tmp_path / "raw.mp4"
97         raw.write_bytes(b"x")
98         calls = []
99
100        def fake_run(cmd, *a, **k):
101            calls.append(cmd)
102            if cmd[0] == "ffprobe":
103                return MagicMock(stdout="100.0\n", returncode=0)
104            out = cmd[-1]
105            if isinstance(out, str) and out.endswith(".mp4"):
106                open(out, "wb").write(b"clip")
107            return MagicMock(returncode=0, stdout=b"", stderr=b"")
108
109        with patch.object(subprocess, "run", side_effect=fake_run):
110            comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
111
112        trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
113        assert trim[trim.index("-preset") + 1] == "superfast"
114        assert trim[trim.index("-crf") + 1] == "22"
115        assert trim[trim.index("-b:a") + 1] == "128k"
116        assert "-vf" not in trim
117
118
119     def test_compile_sd_profile_scales_and_uses_cheaper_encode_args(tmp_path):
120         comp = VideoCompiler(

```

13. assistant (2026-07-08T06:18:31.700Z)

I have enough to reach a verdict. Let me confirm the one remaining factual point ? that NVENC ('h264_nvenc') is only exercised on the L4 worker path and is opt-in everywhere else.

14. user (2026-07-08T06:18:36.281Z)

```

backend/config/models.py:389:     # concat stays -c copy). Default ``libx264`` (CPU, works
everywhere incl. CI). ``h264_nvenc`` /
backend/config/models.py:390:     # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically
faster on the 4K re-encode that
backend/config/models.py:393:     # cloud compile-dispatch forwards
``deployment.cloud_stitch_encoder`` onto this for the L4 run;
backend/config/models.py:395:     encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] =
"libx264"
backend/config/models.py:537:     # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the
4K re-encode the watermark forces;
backend/config/models.py:540:     cloud_stitch_encoder: Literal["libx264", "h264_nvenc",
"hevc_nvenc"] = "h264_nvenc"
backend/orchestration.py:1367:     # (deployment.cloud_stitch_encoder, NVENC by default) so the
worker hardware-encodes the reel.
backend/orchestration.py:1370:     getattr(cfg, "deployment", None), "cloud_stitch_encoder",
"h264_nvenc"

```

15. assistant (2026-07-08T06:18:52.225Z)

I have all the evidence needed to reach a verdict.

****Analysis of the claim:****

The reviewer's **factual** premise is correct: `\_probe\_encoder\_available` (compiler.py:54-55) test-encodes with only `-c:v h264\_nvenc`, while `\_select\_encoder` (compiler.py:358-360) runs the real per-clip encode with `-c:v <enc> -preset p4 -cq <cq> -pix\_fmt <pix\_fmt>`. The args genuinely differ, and if the real encode failed on every clip, `temp\_clips` would be empty and `compile\_highlights` would raise `RuntimeError` (compiler.py:633-639).

But the claimed *failure* is not concretely reachable:

1. **The `-preset p4` (`p1?p7`) naming has been supported since ffmpeg 4.3 (June 2020).** For a bare-NVENC probe to succeed while `-preset p4` is rejected, you need a pre-4.3 ffmpeg that still has a functioning NVIDIA GPU/NVENC ? a build ~6 years old in a 2026 deployment.

2. **NVENC is the *only* path where this could bite, and it exists only on the L4 image, which has a modern ffmpeg.** `StitchingConfig.encoder` defaults to `libx264` (`models.py:395`); `h264_nvenc` is only reached via `deployment.cloud_stitch_encoder` (default `h264_nvenc`, `models.py:540`) forwarded onto the L4 compile-dispatch (`orchestration.py:1367-1370`). The L4 worker image installs ffmpeg from Ubuntu 22.04 apt (`deploy/cloudrun/Dockerfile:47-50`) = ffmpeg 4.4.2, which fully supports `-preset p4`.

3. **Everywhere else the probe correctly protects the fallback:** on a non-GPU/CI box the bare NVENC probe fails, `_probe_encoder_available` returns False, and `_select_encoder` falls back to `libx264` (`compiler.py:334-340`) ? exactly as designed.

The reviewer self-labels this "low confidence" and "version-gated (modern ffmpeg on the L4 image is unaffected)," which concedes the actual deployment target is fine. The scenario relies on a hypothetical misconfiguration (`h264_nvenc` on a pre-2020 ffmpeg with a working GPU) that is not any deployment target and cannot be concretely confirmed. This is a speculative robustness/hardening observation, not a demonstrable defect in the changed code.

16. user (2026-07-08T06:18:59.327Z)

Structured output provided successfully